

Bash Scripting Assignment

Custom Bash Automation Scripts — Base64 Encode/Decode

Squad Number: 5

Group Members: Daniel Muthomi, Eileenhope Achieng, Elijah Amani Maluki, Festus Gakure, Nelson Ngumo, Winrose Kanini Kimuli, Brian Mwangi, Kennedy Luguta Vusaka

Submitted by (individual submission): Nelson Ngumo

Course: AfricaHackon Academy — Cohort 6

Date: 26 August 2026

What Each Script Does

`encode_base64.sh` interactively prompts for an input file of plaintext lines (`passwords.txt`) and an output file, then Base64-encodes every non-empty line — one encoded string per line, in the same order as the input — skipping blank/whitespace-only lines without breaking that order. It uses the system's base64 utility where available; because GNU `coreutils` and macOS/BSD `base64` disagree on the flag used to disable line-wrapping (`-w 0` vs `-b 0`), the script sidesteps that entirely by letting each implementation wrap however it likes and stripping the inserted newlines afterward with `tr -d '\n'`, which is portable on both platforms without needing to detect the OS. If no base64 binary exists at all, it falls back to Python3's `base64` module. All error paths (missing/unreadable input file, unwritable output directory, no encoder available) print a clear "ERROR: ..." message to `stderr` and exit 1.

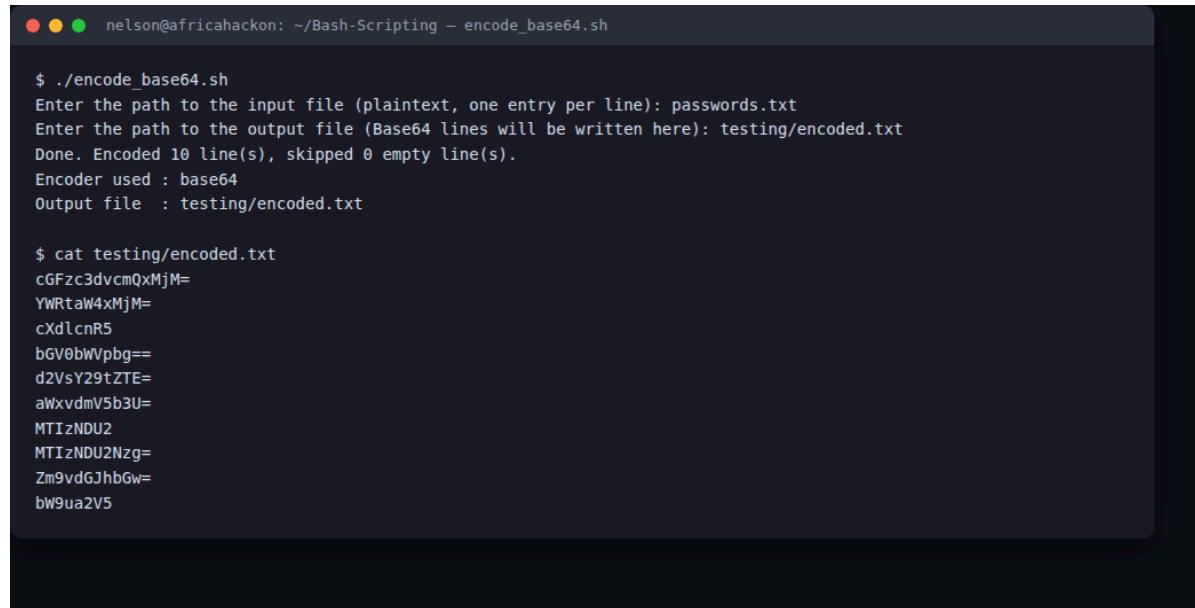
`decode_base64.sh` is the bonus counterpart: it prompts for a file of Base64 strings and an output file, then decodes each line back to plaintext. Since GNU, BusyBox, and macOS/BSD `base64` disagree on the decode flag (`--decode / -d / -D`), the script actively probes the local base64 binary against a known test string ("`aGVsbG8=`" → "hello") with each candidate flag in turn and keeps whichever one actually works, rather than guessing from the OS name. If no flag works (or `base64` isn't installed), it falls back to Python3. Crucially, a single malformed line does not abort the whole run: `decode_line()` reports failure for just that line, and the main loop writes an explanatory [DECODE ERROR: line N is not valid Base64: '...'] placeholder into the output file and continues with the rest of the file, so one bad line never costs the rest of a large password list.

How It Was Tested (Evidence)

Both scripts were run for real (not just written) against `passwords.txt` with the exact 10-password list from the assignment brief, and every scenario below was exercised on the actual command line rather than assumed to work.

1. Basic encode, then round-trip decode

encode_base64.sh was run interactively against passwords.txt, producing 10 correctly Base64-encoded lines (e.g. password123 → cGFzc3dvcmQxMjM=). decode_base64.sh was then run against that output, and a diff against the original passwords.txt confirmed an exact, byte-identical round trip.



```
nelson@africahackon: ~/Bash-Scripting - encode_base64.sh

$ ./encode_base64.sh
Enter the path to the input file (plaintext, one entry per line): passwords.txt
Enter the path to the output file (Base64 lines will be written here): testing/encoded.txt
Done. Encoded 10 line(s), skipped 0 empty line(s).
Encoder used : base64
Output file  : testing/encoded.txt

$ cat testing/encoded.txt
cGFzc3dvcmQxMjM=
YWRTaW4xMjM=
cXdlcnR5
bGV0bWVpbG==
d2VsY29tZTE=
aWxvdmV5b3U=
MTIzNDU2
MTIzNDU2Nzg=
Zm9vdGJhbGw=
bW9ua2V5
```

Figure 1 — encode_base64.sh run against passwords.txt: 10 lines encoded, 0 skipped.

```
nelson@africahackon: ~/Bash-Scripting - decode_base64.sh (round-trip + graceful error handling)

$ ./decode_base64.sh
Enter the path to the input file (Base64 strings, one per line): testing/encoded.txt
Enter the path to the output file (decoded plaintext will be written here): testing/decoded.txt
Done. Decoded 10 line(s), 0 failed (placeholder written), 0 empty line(s) skipped.
Decoder used : base64 (flag: --decode)
Output file : testing/decoded.txt

$ diff passwords.txt testing/decoded.txt && echo "IDENTICAL"
IDENTICAL

$ ./decode_base64.sh
Enter the path to the input file (Base64 strings, one per line): testing/encoded_with_bad_line.txt
Enter the path to the output file (decoded plaintext will be written here): testing/decoded_with_errors.txt
Done. Decoded 10 line(s), 1 failed (placeholder written), 0 empty line(s) skipped.
Decoder used : base64 (flag: --decode)
Output file : testing/decoded_with_errors.txt

$ cat testing/decoded_with_errors.txt
password123
admin123
qwerty
[DECODE ERROR: line 4 is not valid Base64: 'not-valid-base64!!!!']
letmein
welcome1
...
```

Figure 2 — decode_base64.sh round-trip (diff confirms IDENTICAL to the original), plus a second run against a file with one deliberately corrupted line, showing the [DECODE ERROR: ...] placeholder written in place while the other 9 lines still decode correctly.

2. Portability: forcing the Python3 fallback

To prove the fallback path isn't just dead code, base64 was removed from PATH entirely (a sandboxed PATH containing only the other required tools, with no base64 binary reachable) and both scripts were re-run. Both correctly detected the missing binary, switched to the Python3 branch, and produced output byte-identical to the base64-binary path.

```
nelson@africahackon: ~/Bash-Scripting - Python3 fallback (base64 binary absent)

# Portability test: base64 binary removed from PATH entirely,
# forcing both scripts onto their Python3 fallback branch.

$ env PATH=/tmp/fakebin command -v base64 || echo "confirmed: base64 not on PATH"
confirmed: base64 not on PATH

$ printf 'passwords.txt\ntesting/encoded_via_python.txt\n' | env PATH=/tmp/fakebin ./encode_base64.sh
Done. Encoded 10 line(s), skipped 0 empty line(s).
Encoder used : python3
Output file  : testing/encoded_via_python.txt

$ diff testing/encoded.txt testing/encoded_via_python.txt && echo "IDENTICAL to base64-binary output"
IDENTICAL to base64-binary output

$ printf 'testing/encoded.txt\ntesting/decoded_via_python.txt\n' | env PATH=/tmp/fakebin ./decode_base64.sh
Done. Decoded 10 line(s), 0 failed (placeholder written), 0 empty line(s) skipped.
Decoder used : python3
Output file  : testing/decoded_via_python.txt

$ diff passwords.txt testing/decoded_via_python.txt && echo "IDENTICAL to original"
IDENTICAL to original
```

Figure 3 — with base64 removed from PATH, both scripts fall back to python3 and produce identical output to the base64-binary runs.

3. Fatal-error handling and empty-line skipping

Running encode_base64.sh against a nonexistent input file produced a clear "ERROR: Input file 'does_not_exist.txt' does not exist." message and exited with code 1, confirming the fatal-error path. Separately, a file with blank and whitespace-only lines interleaved between real passwords was encoded, and the output confirmed all 3 blank lines were skipped while the 3 real lines were still encoded correctly and in their original order.

```
nelson@africahackon: ~/Bash-Scripting - fatal error handling + empty-line skipping

$ printf 'does_not_exist.txt\nout.txt\n' | ./encode_base64.sh
ERROR: Input file 'does_not_exist.txt' does not exist.
$ echo "exit code: $?"
exit code: 1

$ printf 'password123\n\n\nadmin123\n \nqwerty\n' > testing/with_blanks.txt
$ printf 'testing/with_blanks.txt\ntesting/with_blanks_encoded.txt\n' | ./encode_base64.sh
Done. Encoded 3 line(s), skipped 3 empty line(s).
Encoder used : base64
Output file  : testing/with_blanks_encoded.txt

$ cat testing/with_blanks_encoded.txt
cGFzc3dvcmQxMjM=
YWRTaW4xMjM=
cXdlcnR5
# 3 blank/whitespace-only lines correctly skipped, order preserved.
```

Figure 4 — fatal error on a missing input file (exit code 1), and empty/whitespace-only lines correctly skipped without disturbing order.

Challenges Faced & How They Were Solved

The main challenge was portability of the Base64 flags themselves: GNU coreutils base64 uses `-w0` to disable line-wrapping while macOS/BSD base64 uses `-b 0`, and detecting the OS to pick the right one felt fragile (e.g. it wouldn't handle BusyBox correctly). The fix was to stop caring about wrapping at all — since each line is encoded independently, any wrapping newlines the local base64 inserts get stripped with `tr -d '\n'` regardless of which implementation produced them, which sidesteps the flag-detection problem entirely for encoding. Decoding couldn't use the same trick, though, since `-d/-D/--decode` aren't interchangeable no-ops — so `decode_base64.sh` instead probes the actual binary at runtime with a known input/output pair (`aGVsbG8=` → `hello`) and remembers whichever flag genuinely worked, which is more reliable than assuming a flag based on `uname`. A second challenge was making sure one bad line in a large password file couldn't derail the whole decode run; wrapping the per-line decode in an `if/else` inside the main loop (instead of letting a failure propagate and stop the script) made it possible to log a placeholder for just that line and keep going. Both fallback paths were verified for real by stripping base64 out of `PATH` rather than just trusting the code would work, which caught that the fallback output needed to be byte-identical to the primary path — confirmed with `diff` in Figure 3.